CS316 Chapter 2.3: Handling Missing Data

Slides + Gradebook Rescue notebook, walked through in order, with traps

PART 1: SLIDES

Slide 2: Why handle missing data, and the options

- Missing data distorts analysis and leads to **biased results**.
- Proper handling ensures data integrity and better model performance.
- In pandas a missing value appears as **NaN** (np.nan). None is also treated as missing.

Option	When	Impact
Drop	Missing values are few or non-critical	Reduces dataset size, but avoids introducing bias
Fill (impute)	Replace with mean, median or a constant	Retains all data, but assumptions must be valid to avoid bias
Forward/backward fill	Use prior or next data points	Best for time series or sequential data

Slides 3-4: Identify and count

```
df.isnull()          # True = missing, False = present (same as df.isna())
df.isnull().sum()    # missing count per column
# Slide output: name 5, major01 0, major02 5, project 3, final 4
```

Slides 5-6: Fill

```
df['final'].fillna(0, inplace=True)                          # constant: NaN -> 0
df['major02'].fillna(df['major02'].mean(), inplace=True)     # mean: Ahmed's NaN -> 11.73
```

Slides 7-8: Drop

```
df.dropna(inplace=True)                      # rows with ANY missing value
df.dropna(subset=['final'], inplace=True)    # only rows missing 'final'
df.dropna(axis=0, inplace=True)              # drop rows
df.dropna(axis=1, inplace=True)              # drop columns
# thresh= keeps rows/columns with a minimum number of NON-null values
```

Slide 9: Full example

```
df = pd.DataFrame({'A': [1, 2, None, 4],
                   'B': [None, 2, 3, 4],
                   'C': [1, None, 3, 4]})
df.isnull()            # detect
df.dropna()            # only row 3 remains (the only complete row)
df.fillna(value=0)     # every NaN becomes 0
df.fillna(df.mean())   # A: 2.33, B: 3.0, C: 2.67 (each column's OWN mean)
```

PART 2: NOTEBOOK (Gradebook Rescue)

Scenario: you are the TA. The CS316 gradebook export has blank cells, grades are due in 48 minutes, and the instructor says: *"Don't invent grades that aren't real."*

Step 0: Setup

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)   # show all columns (no limit)
pd.set_option("display.width", 120)          # print up to 120 characters wide
sns.set_style("whitegrid")                   # white background with grid lines
```

Step 1: Load the data

```
df = pd.DataFrame(data)   # 20 students: student_id, name, major01, major02, project, final
df.shape                  # (20, 6)
df.info()                 # non-null count per column (missing = rows - non-null)
```

Step 2: Assess the damage

```
df.isnull().head(6)                                      # True/False table
missing_counts = df.isnull().sum()                       # count per column
missing_pct = (df.isnull().sum() / len(df) * 100).round(1) # % per column
summary = pd.DataFrame({"missing_count": missing_counts, "missing_pct": missing_pct})
summary["missing_count"].plot(kind="bar", ax=axes[0])     # bar: missing per column
sns.heatmap(df.isnull(), cbar=False, ax=axes[1])          # map: red cell = missing
```

Column	Missing	%
name	5	25%
major01	0	0%
major02	6	30% (most affected)
project	5	25%
final	5	25%
Whole table: df.isnull().sum().sum()	21	

Why .sum() counts missing values: True counts as 1 and False as 0. One .sum() gives a count per column; .sum().sum() gives one total for the whole table. .isnull().mean()*100 gives the % missing.

Why is it missing? (MCAR, MAR, MNAR)

Type	Missing because of...	Example	Effect on handling
MCAR: completely at random	Nothing: pure chance	A system glitch dropped a random cell	Safe to drop or fill
MAR: at random	Another VISIBLE column	Late registrants are missing major01	Filling with the overall mean can be off; fill per group. Blind dropping biases
MNAR: not at random	The MISSING VALUE itself	A student who failed the final did not submit it	Filling with the mean HIDES the problem cases; dropping removes them. Investigate

Step 3: Choose a strategy per column

Column	Strategy	Why
name	Drop rows	No value can substitute for an identity
major01	Nothing	Already complete
major02	Fill with MEAN	Roughly symmetric (mean 11.20 vs median 11.95)
project	Fill with MEDIAN	Skewed by a few low scores (mean 13.69 vs median 14.67)
final	Drop rows	Highest-stakes score; guessing it is not defensible

Strategy A: Drop with dropna()

```
df.dropna()                              # 20 -> 6 rows   (blunt: loses 70% of the class)
df.dropna(subset=["name", "final"])      # 20 -> 12 rows  (targeted: only defensible drops)
df.dropna(thresh=4).shape                # (19, 6): keep rows with at least 4 NON-missing values
df.dropna(axis=1)                        # only student_id and major01 survive
df.dropna(how="all")                     # drop a row only if EVERY value is missing
```

Strategy B: Fill with a constant

```
demo = df[["name", "project"]].copy()
demo["project"] = demo["project"].fillna(0)   # reassign: fillna returns a NEW object
```

Only when the blank has a real known meaning (e.g. a late_penalty blank = no penalty = 0). Putting 0 for a missing exam score would be unfair to the student.

Strategy C: Mean or median

```
df["major02"].mean(), df["major02"].median()   # 11.20 vs 11.95 -> close -> symmetric -> MEAN
df["project"].mean(), df["project"].median()   # 13.69 vs 14.67 -> mean lower -> skewed -> MEDIAN
df_step["major02"] = df_step["major02"].fillna(df_step["major02"].mean())
```

```
df_step["project"] = df_step["project"].fillna(df_step["project"].median())
```

The notebook's histograms draw the mean and median as vertical lines (axvline) to show the gap visually.

Strategy D: Forward / backward fill (ordered data only)

```
attendance["students_present"]           # [42, 40, NaN, NaN, 39, NaN, 41, 41]
attendance["students_present"].ffill()   # [42, 40, 40, 40, 39, 39, 41, 41]  previous value forward
attendance["students_present"].bfill()   # [42, 40, 39, 39, 39, 41, 41, 41]  next value backward
```

Step 4: The cleaning pipeline

```
def clean_gradebook(raw):
    clean = raw.copy()                                          # protect the original
    clean = clean.dropna(subset=["name", "final"])              # drop what can't be guessed
    clean["major02"] = clean["major02"].fillna(clean["major02"].mean())     # symmetric
    clean["project"] = clean["project"].fillna(clean["project"].median())   # skewed
    return clean.reset_index(drop=True)                         # renumber 0, 1, 2...

clean_df = clean_gradebook(df)
assert clean_df.isnull().sum().sum() == 0, "Still missing data"   # error if any NaN remain
```

- **reset_index(drop=True)**: dropna leaves gaps in the index (0, 1, 3, 4...). This renumbers; drop=True means the old index is not kept as a column.
- **assert**: if the condition is False, execution stops with the message.

Exercises: Section B (8 students)

```
section_b.isnull().sum()                    # quiz_avg 3 -> most affected
sb = section_b.dropna(subset=["name"]).copy()
sb["final"]    = sb["final"].fillna(sb["final"].median())
sb["quiz_avg"] = sb["quiz_avg"].fillna(sb["quiz_avg"].mean())
sb["midterm"]  = sb["midterm"].fillna(sb["midterm"].median())
sb.isnull().sum().sum()                     # 0
len(section_b.dropna())                     # 0 -> blunt drop removes EVERY student
```

PART 3: CHOOSING THE STRATEGY

Situation	Use	Why
Symmetric / normal (mean close to median)	Mean	Represents the center well
Spread unknown	Mean	Default choice
Skewed (left or right), outliers, high variance	Median	Not pulled by extreme values
Blank has a known meaning	Constant	Recovers the true value
Must respect a domain boundary	Min / max	Keeps values within known limits
Ordered / time data	ffill / bfill	Neighbours are the best guess
Identity or high-stakes values	Drop	Cannot be guessed honestly
Most of a column missing (60-80%)	Drop the column (axis=1)	Filling would invent most of the data
Sensitive data (e.g. patients)	Drop critical values or get them from the source; median for supporting numbers	The more a value drives a decision, the less acceptable it is to guess it

Clue	Right-skewed	Left-skewed	Symmetric
Mean vs median	Mean > median	Mean < median	Mean about equal to median
Histogram	Long tail to the right	Long tail to the left	Bell shape
Box plot	Median near Q1, long right whisker	Median near Q3, long left whisker	Median centered

Clue	Right-skewed	Left-skewed	Symmetric
Example	Income, house prices	Easy exam scores, project with a few 0s	Height

Example: [10, 12, 11, 13, 100] gives mean 29.2 (pulled up by 100) but median 12 (typical).

PART 4: TRAPS

1. Forgetting to save the result

df.dropna() and df["col"].fillna(0) do NOT change df. Fix: df = df.dropna(), df["col"] = df["col"].fillna(0), or inplace=True.

2. thresh counts NON-missing values

6 columns, 2 missing = 4 real values. thresh=4 keeps the row; thresh=5 drops it.

3. Blunt dropna() destroys data

Gradebook 20 -> 6 rows; Section B 8 -> 0 rows. Prefer subset=[...].

4. NumPy does not skip NaN; pandas does

np.mean([1, nan, 3]) = nan; np.nanmean(...) = 2.0; pd.Series([1, nan, 3]).mean() = 2.0.

5. len vs count

len(s) includes NaN; s.count() counts only non-missing values.

6. NaN is never equal to anything

np.nan == np.nan is False, so df[df["col"] == np.nan] finds nothing. Use df[df["col"].isnull()].

7. ffill / bfill leave NaN at the edges

[NaN, 5, NaN, 8, NaN].ffill() = [NaN, 5, 5, 8, 8]; .bfill() = [5, 5, 8, 8, NaN].

8. dropna does not renumber the index

After dropping, df.loc[2] may raise a KeyError. Fix: reset_index(drop=True).

9. A numeric column with NaN becomes float

pd.Series([1, 2, None]) has dtype float64.

10. Filling MNAR data with the mean

Hides the cases that matter (failing students, very sick patients).

11. ffill on unordered data

In a gradebook, row 5 has nothing to do with row 4; copying values is meaningless.

12. Filling a mostly empty column

Invents most of the data. Drop the column instead.

13. axis direction

axis=0 = rows (default); axis=1 = columns.

14. Mean for skewed data

One extreme value drags the mean; use the median.